LEVEL 11 — Searching

The goal here is not merely to implement search. You should understand **what is being searched, what assumptions the algorithm makes, how the three languages represent the data, and how library functions differ from implementing the algorithm yourself.**

11A — Linear Search

Q71 — Find an Element

Given:

[10, 25, 7, 42, 18, 31]

and a target:

42

find its position.

Expected:

Index = 3

First requirement

Do it **without any library search function**.

Create:

linearSearch(array, target)

Q72 — Element Not Found

Modify Q71.

Test:

target = 100

Decide what your function should return.

Possible choices:

-1
null
None
false
exception

Choose the idiomatic approach for each language.

Compare

C: What is the natural return value?

Java: Should you return `-1` or something else?

Python: What would a function returning an index normally do?

11B — First, Last and All Occurrences

Q73 — First Occurrence

Given:

[4, 7, 2, 7, 9, 7, 1]

find the first occurrence of `7`.

Output:

1

Q74 — Last Occurrence

Find the last occurrence.

Output:

5

Create:

firstIndex()
lastIndex()

Q75 — Count Occurrences

Given:

[4,7,2,7,9,7,1]

find:

target = 7

Output:

3

Q76 — Find All Occurrences

Output:

[1,3,5]

Language comparison

Investigate how you would naturally represent the result in:

- C
- Java
- Python

Think about:

fixed-size array
dynamic array
ArrayList
list

11C — Search in Strings

Q77 — Search for a Character

Given:

"programming"

find:

'g'

First implement the search yourself.

Then compare it with the library/API approaches you investigated in Level 9.

Q78 — Search for a Word

Given:

"I love programming and I love solving problems."

search for:

"programming"

Determine:

- Is it present?
- First position?
- Last position?
- Number of occurrences?

Compare:

C

strstr()

Java

contains()
indexOf()
lastIndexOf()

Python

in
find()
rfind()
count()

11D — Linear Search with Conditions

Q79 — Search for First Even Number

Given:

[11, 17, 23, 8, 19, 20]

find the first even number.

Output:

8

Q80 — Search for First Number Greater Than X

Given:

[4, 8, 12, 3, 17, 2]

and:

X = 10

return:

12

Then modify the function:

findFirst(array, condition)

This becomes an important conceptual bridge toward **callbacks/function objects/higher-order functions**.

Later we will revisit this:

- C → function pointers
- Java → interfaces/lambdas
- Python → functions as objects

11E — Binary Search

Before coding:

Q81 — Can Binary Search Be Used?

For each array, answer **YES or NO**:

[1,2,3,4,5,6,7]
[7,3,1,9,2,5]
[10,20,30,40,50]
[10,10,20,30,30,40]

Explain why.

11F — Basic Binary Search

Q82

Given:

[2,5,8,12,16,23,38,56,72]

find:

23

Implement:

binarySearch(array, target)

using an **iterative** approach.

Q83 — Trace the Algorithm

For:

[2,5,8,12,16,23,38,56,72]

search for:

23

Write every iteration:

low
high
mid
array[mid]
comparison
next range

Do this by hand before running your program.

11G — Binary Search Edge Cases

Q84

Test:

[]
[1]
[1,2]
[1,2,3]

Search for:

-1
0
1
2
3
100

Your implementation should not crash.

Q85 — First Occurrence with Binary Search

Given:

[1,2,2,2,3,4,5]

find the **first occurrence** of 2.

Expected:

1

Don't simply stop when you find 2.

Continue searching the appropriate half.

Q86 — Last Occurrence with Binary Search

Same array:

[1,2,2,2,3,4,5]

find the last occurrence.

Expected:

3

11H — Binary Search Variations

Q87 — Lower Bound

Given:

[1,3,5,7,9]

find the first position where:

array[index] >= target

Test:

target = 0
target = 1
target = 4
target = 5
target = 10

Q88 — Upper Bound

Find the first position where:

array[index] > target

Use:

[1,2,2,2,3,4]

and targets:

1
2
3
4
5

This is an important stepping stone toward more advanced searching later.

11l — Search in a Sorted String/Collection

Q89

Given an alphabetically sorted collection:

["apple", "banana", "grape", "mango", "orange"]

search for:

"mango"

Implement binary search for strings.

Then test:

"apple"
"orange"
"pear"

Compare

How does comparison differ between:

C

strcmp()

Java

compareTo()

Python

< > ==

?

11J — Library Search Investigation

Now stop implementing everything manually.

Q90 — Find the language's built-in search tools

Investigate the appropriate library functions/methods for searching.

C

Investigate:

<string.h>
  strchr()
  strrchr()
  strstr()

<stdlib.h>
  bsearch()

For `bsearch()`, determine:

- arguments
- return value
- comparison function
- ordering requirement
- what happens if the element isn't found

This is your first serious introduction to a **function pointer used as an argument**.

Java

Investigate:

java.util.Arrays

particularly:

Arrays.binarySearch()

Test different overloads.

Investigate:

binarySearch(array, key)
binarySearch(array, fromIndex, toIndex, key)

Then determine what the return value means when the element is not found.

Also investigate:

Collections.binarySearch()

and understand when you would use it.

Python

Investigate:

bisect

especially:

bisect.bisect_left()
bisect.bisect_right()
bisect.insort_left()
bisect.insort_right()

Test:

[1,2,2,2,3,4]

with:

2
3
5
0

Important comparison

Compare:

C bsearch()
Java Arrays.binarySearch()

Python bisect

Ask:

> Are they actually solving exactly the same problem?

LEVEL 12 — Sorting

Now we move from **finding things** to **organizing things**.

12A — Basic Sorting

Q91 — Sort Ascending

Given:

[5,2,8,1,9,3]

produce:

[1,2,3,5,8,9]

Do **not** use a library function.

12B — Bubble Sort

Q92

Implement:

bubbleSort(array)

For:

[5,1,4,2,8]

trace every pass.

Record:

Pass 1
Pass 2
Pass 3
...

Q93 — Bubble Sort Optimization

Modify bubble sort so that it stops if a complete pass makes **no swaps**.

Test:

[1,2,3,4,5]

Compare the number of comparisons/swaps with:

[5,4,3,2,1]

12C — Selection Sort

Q94

Implement:

selectionSort(array)

For:

[64,25,12,22,11]

Trace:

minimum position
swap position
array after each pass

12D — Insertion Sort

Q95

Implement:

insertionSort(array)

Test:

[5,2,4,6,1,3]

Trace:

key
previous elements
shifts
final insertion position

12E — Compare the Three

Q96 — Sorting Laboratory

Run:

bubbleSort()
selectionSort()
insertionSort()

on:

Case 1

[1,2,3,4,5]

Case 2

[5,4,3,2,1]

Case 3

[3,1,4,2,5]

Case 4

[7,7,7,7,7]

Case 5

A large randomly generated array.

Measure:

- comparisons

- swaps
- execution time

Don't worry about sophisticated benchmarking yet.

12F — Sorting in Reverse

Q97

Sort:

[5,2,8,1,9,3]

descending.

First modify your own algorithm.

Then investigate how the language's library supports custom ordering.

This is where the three languages become interesting.

12G — C Sorting

Q98 — Investigate `qsort()`

Header:

<stdlib.h>

Investigate:

qsort()

You must understand all four arguments.

Conceptually:

```
qsort(
    array,
    number_of_elements,
    size_of_each_element,
```

    comparison_function
)

Then implement ascending integer sorting.

Extension

Change the comparison function to:

- descending
- absolute value
- odd numbers first
- even numbers first

This will prepare you for **function pointers**.

12H — Java Sorting

Q99 — Investigate Java Sorting APIs

Investigate:

java.util.Arrays

Methods such as:

Arrays.sort()

Then investigate the relevant overloads for:

- primitive arrays
- object arrays
- ranges
- custom comparators

Then investigate:

java.util.Collections

and:

Collections.sort()

Then investigate:

Comparator

12I — Python Sorting

Q100 — Sorting Investigation

Compare:

sorted()

and:

list.sort()

Investigate:

key=
reverse=

Test:

sorted(numbers)sorted(numbers, reverse=True)

Then:

sorted(words, key=len)

Then:

sorted(words, key=str.lower)

12J — Custom Sorting

Q101 — Sort by Multiple Criteria

Given:

```
[
    ("Samad", 80),
    ("Ali", 95),
    ("John", 80),
    ("David", 70)
]
```

Sort by:

1. Marks ascending
2. Marks descending
3. Name alphabetically
4. Marks descending, then name ascending

Solve in:

- C
- Java
- Python

This is an important language comparison.

C

Think:

qsort()
function pointer

Java

Think:

Comparator
Comparator.comparing()
thenComparing()

Python

Think:

key=
lambda
tuple keys

LEVEL 13 — Recursion

Now we revisit problems you've already solved iteratively.

The goal is to understand **when recursion naturally represents the problem**, not to use recursion everywhere.

13A — Factorial

Q102

Implement:

factorial(n)

both:

iterative
recursive

Test:

0
1
5
10

Then investigate what happens with increasingly large n.

13B — Sum of Numbers

Q103

Calculate:

1 + 2 + 3 + ... + N

using recursion.

Then:

sum(array)

recursively.

13C — Reverse a String

Q104

Reverse:

"hello"

using recursion.

Then explain:

- What is the base case?
- What becomes smaller at every recursive call?

13D — Recursive Fibonacci

Q105

Implement:

fib(n)

recursively.

Test:

fib(0)
fib(1)
fib(2)
fib(5)
fib(10)
fib(20)

Then investigate why:

fib(40)

is dramatically slower.

Don't optimize it yet.

We want you to **see the problem first**.

13E — Recursive Digit Problems

Q106

Using recursion:

sumDigits(n)
countDigits(n)
reverseNumber(n)

Then compare these with your Level 3 implementations.

13F — Recursion and Language Behavior

Q107

Create a recursive function that calls itself until `N = 0`.

Test a very large `N`.

Observe:

C

What happens to the call stack?

Java

What exception/error occurs?

Python

Investigate Python's recursion limit.

Look up:

sys.getrecursionlimit()

and:

sys.setrecursionlimit()

Do not blindly increase the limit. The purpose is to understand the runtime behavior.

LEVEL 14 — Linked Lists

Now we officially enter data structures.

We'll begin with the simplest possible linked list.

Q108 — Create One Node

C

Create:

struct Node

containing:

data
next

Create one node dynamically.

Java

Create:

class Node

with:

int data
Node next

Python

Create:

class Node

with:

data

next

Q109 — Two Nodes

Create:

10 → 20 → NULL

Print:

10
20

Do this separately in C, Java and Python.

Q110 — Linked List Traversal

Create:

10 → 20 → 30 → 40 → NULL

Write:

display()

Q111 — Insert at Beginning

Starting with:

20 → 30 → 40

insert:

10

Result:

10 → 20 → 30 → 40

Q112 — Insert at End

Starting with:

10 → 20 → 30

insert:

40

Q113 — Insert at Position

Given:

10 → 20 → 40 → 50

insert `30` at position `2`.

Result:

10 → 20 → 30 → 40 → 50

Test:

- position 0
- beginning
- middle
- end
- after end
- empty list

Q114 — Delete

Implement:

deleteBeginning()
deleteEnd()
deleteValue()
deletePosition()

Test all edge cases.

Q115 — Search Linked List

Implement:

contains(value)
find(value)

Then compare linked-list searching with array searching.

Q116 — Reverse Linked List

Given:

10 → 20 → 30 → 40 → NULL

produce:

40 → 30 → 20 → 10 → NULL

First solve iteratively.

Then recursively.

Q117 — Linked List Library/Language Comparison

Investigate the standard collection APIs rather than assuming your own linked list is always what the language uses.

C

Understand that you generally construct the structure yourself using pointers and memory allocation.

Java

Investigate:

java.util.LinkedList

and compare it with:

ArrayList

Python

Investigate:

collections.deque

and understand why Python doesn't provide a built-in general-purpose `LinkedList` equivalent in the same way Java does.

This is another important lesson:

> **Knowing a data structure and knowing when a language's standard library provides a better abstraction are two different skills.**

After Level 14

The progression will continue:

Level 15 — Stacks

- array implementation
- linked-list implementation
- C
- Java
- Python
- library stacks/deques
- parentheses
- expression problems
- undo-like behavior

Level 16 — Queues

- normal queue
- circular queue
- deque
- priority queue
- implementations vs standard libraries

Level 17 — Hashing

- hash tables
- frequency problems
- collisions

- maps/dictionaries
- C implementation
- Java `HashMap`
- Python `dict`

Level 18 — Trees

- binary trees
- traversal
- BST
- insertion/deletion/search
- recursion

Level 19 — Heaps / Priority Queues

Level 20 — Basic Graphs

- adjacency matrix
- adjacency list
- BFS
- DFS